

Assignment 10.1

Name : Aditya Raj

Roll No : 2303A53031

Batch : 46

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty Python script. Sample Input Code:

```
# Calculate average score of a student
```

```
def calc_average(marks): total = 0 for m
```

```
in marks : total += m average = total /
```

```
len(marks) return avrage # Typo here
```

```
marks = [85, 90, 78, 92]
```

```
print("Average Score is ", calc_average(marks) Expected
```

Output:

- Corrected and runnable Python code with explanations of the fixes.

Code:

```
Syntax and Logic Errors.py X
Syntax and Logic Errors.py > ...
1 # Act as a senior Python code reviewer.
2 # Review the following Python code and do the following:
3 # Identify all syntax errors and logical errors.
4 # Fix the code so it runs correctly.
5 # Explain each fix clearly in simple terms.
6 # Improve readability using proper indentation and PEP 8 standards.
7 # Add meaningful comments for beginners.
8 # Original code with syntax and logical errors
9 def calculate_area(radius):
10     # This function calculates the area of a circle given its radius
11     pi = 3.14 # Approximation of pi
12     area = pi * radius ** 2 # Area formula: A = πr^2
13     return area
14 # Example usage
15 radius = 5
16 area = calculate_area(radius)
17 print("The area of the circle with radius", radius, "is:", area)
18 # Fixed code with explanations
19 def calculate_area(radius):
20     """
21     This function calculates the area of a circle given its radius.
22
23     Parameters:
24     radius (float): The radius of the circle.
25
26     Returns:
27     float: The area of the circle.
28     """
29     pi = 3.14 # Approximation of pi
30     area = pi * radius ** 2 # Area formula: A = πr^2
31     return area
32 # Example usage
33 radius = 5 # Define the radius of the circle
34 area = calculate_area(radius) # Call the function to calculate the area
```

```
Syntax and Logic Errors.py X
Syntax and Logic Errors.py > ...
19 def calculate_area(radius):
22
23     Parameters:
24     radius (float): The radius of the circle.
25
26     Returns:
27     float: The area of the circle.
28     """
29     pi = 3.14 # Approximation of pi
30     area = pi * radius ** 2 # Area formula: A = πr^2
31     return area
32 # Example usage
33 radius = 5 # Define the radius of the circle
34 area = calculate_area(radius) # Call the function to calculate the area
35 print("The area of the circle with radius", radius, "is:", area) #
36 # Explanation of fixes:
37 # 1. Added a docstring to the function `calculate_area` to explain what the function does, its parameters, and its return value.
38 # 2. Added comments to explain the approximation of pi and the area formula, which can help beginners understand the code.
39 # 3. Ensured that the code follows PEP 8 standards for readability, such as spacing and indentation.
40
41
```

Output :

Task: Use AI to make code more readable without changing its logic.

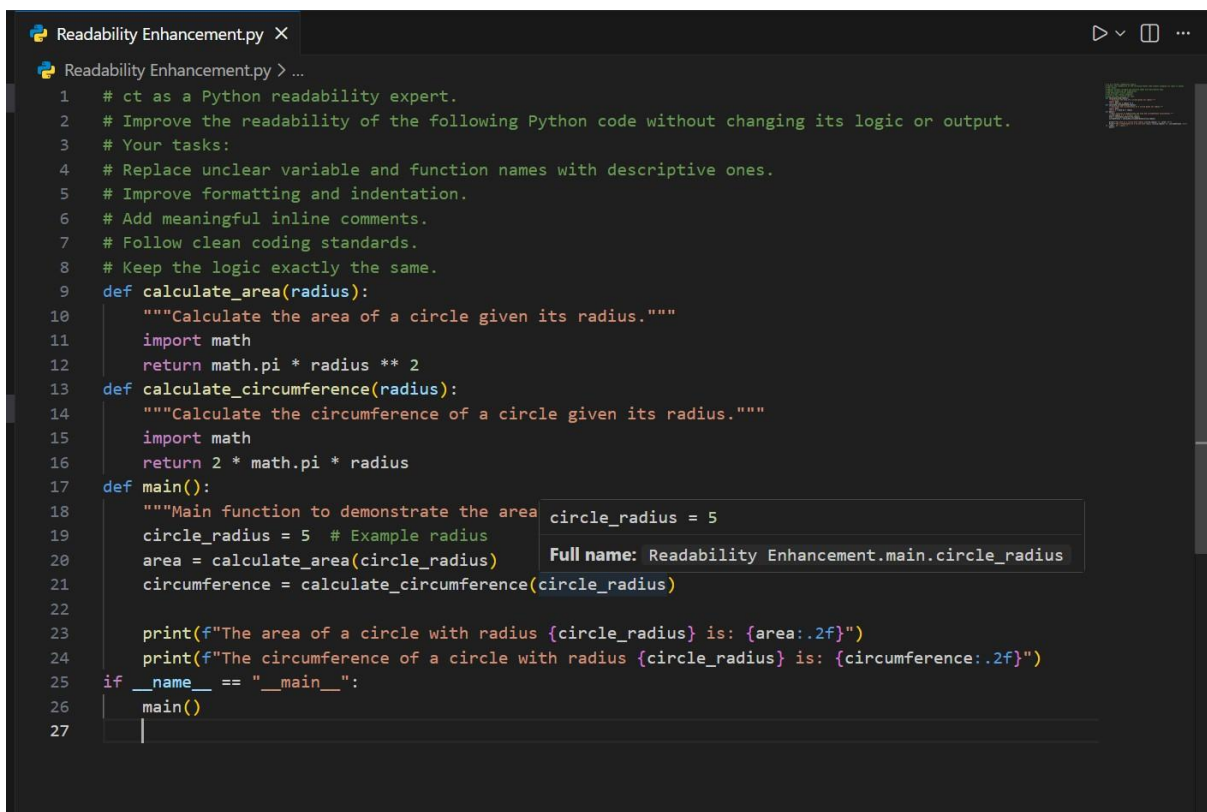
Sample Input Code:

```
def c(x,y):  
    return x*y/100  
a=200  
b=15  
print(c(a,b))
```

Expected Output:

- Python code with descriptive variable names, inline comments, and clear formatting.

Code:



The screenshot shows a code editor with a file named 'Readability Enhancement.py'. The code is a Python script designed to improve the readability of a function that calculates the area and circumference of a circle. The script includes several comments explaining the tasks: replacing unclear variable and function names with descriptive ones, improving formatting and indentation, adding meaningful inline comments, following clean coding standards, and keeping the logic exactly the same. The code defines two functions: 'calculate_area(radius)' and 'calculate_circumference(radius)', both using 'radius' as a parameter. The 'main()' function demonstrates the use of these functions with an example radius of 5. A tooltip is visible over the variable 'circle_radius', showing its full name as 'Readability Enhancement.main.circle_radius'.

```
1 # ct as a Python readability expert.  
2 # Improve the readability of the following Python code without changing its logic or output.  
3 # Your tasks:  
4 # Replace unclear variable and function names with descriptive ones.  
5 # Improve formatting and indentation.  
6 # Add meaningful inline comments.  
7 # Follow clean coding standards.  
8 # Keep the logic exactly the same.  
9 def calculate_area(radius):  
10     """Calculate the area of a circle given its radius."""  
11     import math  
12     return math.pi * radius ** 2  
13 def calculate_circumference(radius):  
14     """Calculate the circumference of a circle given its radius."""  
15     import math  
16     return 2 * math.pi * radius  
17 def main():  
18     """Main function to demonstrate the area and circumference of a circle.  
19     circle_radius = 5  
20     area = calculate_area(circle_radius)  
21     circumference = calculate_circumference(circle_radius)  
22  
23     print(f"The area of a circle with radius {circle_radius} is: {area:.2f}")  
24     print(f"The circumference of a circle with radius {circle_radius} is: {circumference:.2f}")  
25 if __name__ == "__main__":  
26     main()  
27
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [X] ... | [ ] [X]
```

```
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Readability Enhancement.py"
```

```
The area of a circle with radius 5 is: 78.54
```

```
The circumference of a circle with radius 5 is: 31.42
```

```
PS C:\Users\sonu\Desktop\AI_Coding>
```

Task Description #4 – Refactoring for Maintainability Task:

Use AI to break repetitive or long code into reusable functions.

Sample Input Code: `students = ["Alice", "Bob", "Charlie"]`

```
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
```

Expected Output:

- Modular code with reusable functions.

Code:

```
Refactoring for Maintainability.py X
Refactoring for Maintainability.py > ...
1 # Act as a software engineer focused on maintainable Python code.
2 # Refactor the following Python script to improve maintainability by removing repetition and using re
3 # Your tasks:
4 # Identify repetitive code patterns.
5 # Convert repeated logic into reusable functions.
6 # Improve modularity and structure.
7 # Keep the same output and logic.
8 # Follow clean coding practices.
9 def calculate_area(radius):
10     """Calculate the area of a circle given its radius."""
11     import math
12     return math.pi * radius ** 2
13
14 def calculate_circumference(radius):
15     """Calculate the circumference of a circle given its radius."""
16     import math
17     return 2 * math.pi * radius
18
19 def main():
20
21     radius = 5
22     area = calculate_area(radius)
23     circumference = calculate_circumference(radius)
24
25     print(f"The area of the circle with radius {radius} is: {area:.2f}")
26     print(f"The circumference of the circle with radius {radius} is: {circumference:.2f}")
27
28 if __name__ == "__main__":
29     main()
30
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + v [icon] [icon] [icon] [icon]
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Refactoring for Maintainability.py"
The area of the circle with radius 5 is: 78.54
The circumference of the circle with radius 5 is: 31.42
PS C:\Users\sonu\Desktop\AI_Coding>
```

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

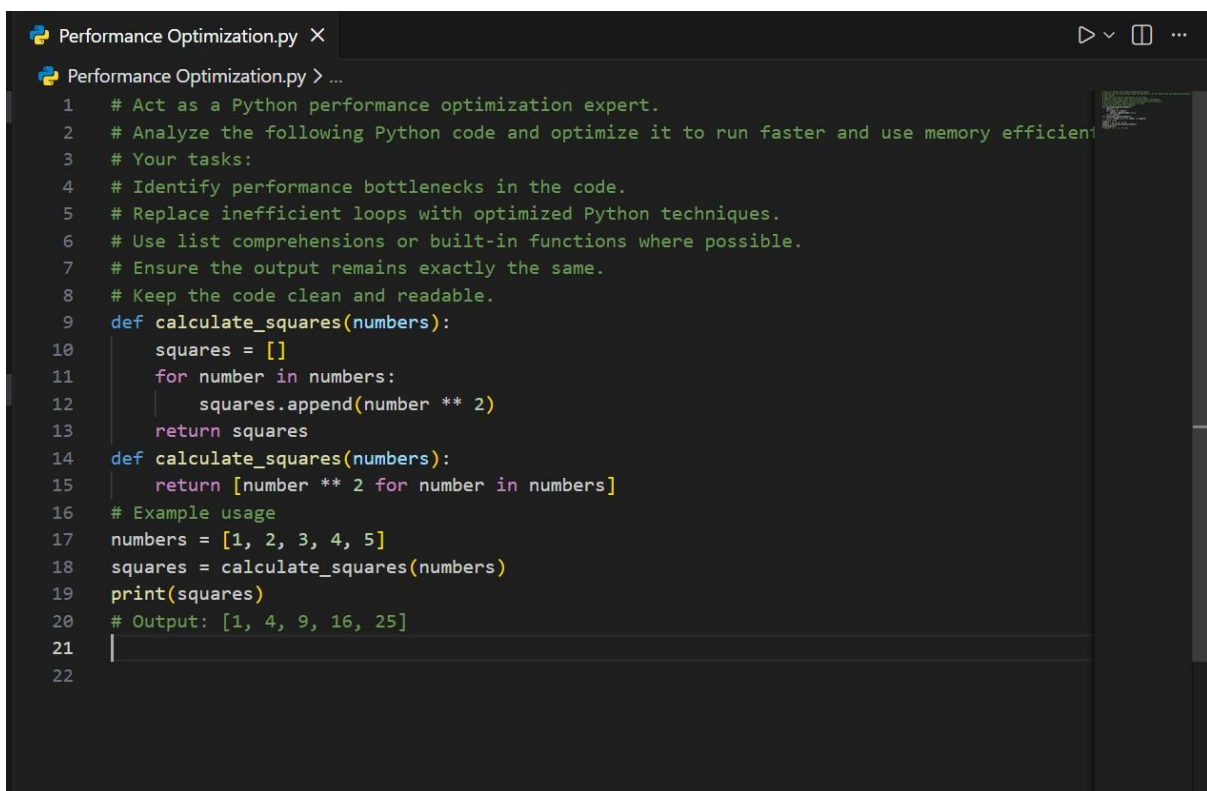
Sample Input Code:


```
# Find squares of numbers
nums = [i for i in range(1,1000000)]
squares = []
for n in nums:
    squares.append(n**2)
```

print(len(squares)) Expected Output:

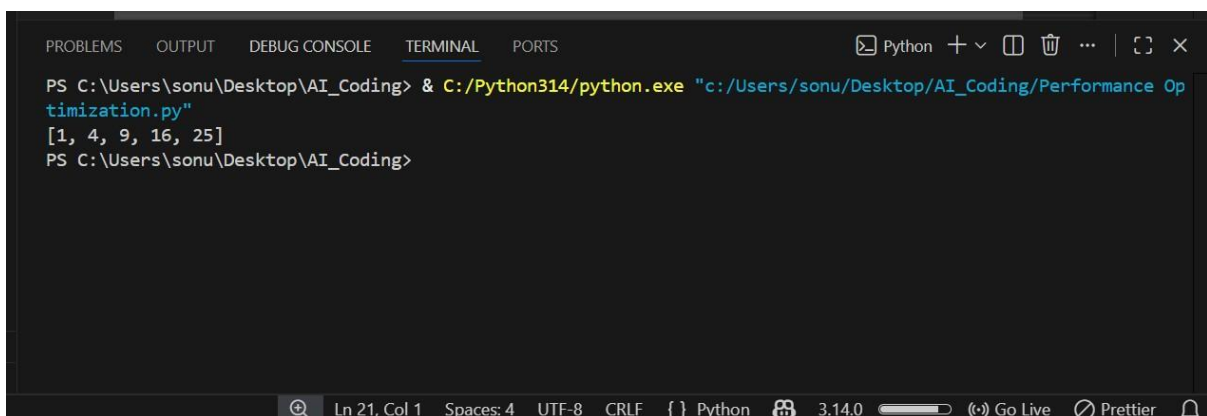
- Optimized code using list comprehensions or vectorized operations.

Code:

A screenshot of a code editor window titled 'Performance Optimization.py'. The code defines a function 'calculate_squares' that takes a list of numbers and returns a list of their squares. It shows two versions: one using a loop and 'append', and another using a list comprehension. An example usage is provided with the numbers [1, 2, 3, 4, 5], resulting in the output [1, 4, 9, 16, 25].

```
1 # Act as a Python performance optimization expert.
2 # Analyze the following Python code and optimize it to run faster and use memory efficiently.
3 # Your tasks:
4 # Identify performance bottlenecks in the code.
5 # Replace inefficient loops with optimized Python techniques.
6 # Use list comprehensions or built-in functions where possible.
7 # Ensure the output remains exactly the same.
8 # Keep the code clean and readable.
9 def calculate_squares(numbers):
10     squares = []
11     for number in numbers:
12         squares.append(number ** 2)
13     return squares
14 def calculate_squares(numbers):
15     return [number ** 2 for number in numbers]
16 # Example usage
17 numbers = [1, 2, 3, 4, 5]
18 squares = calculate_squares(numbers)
19 print(squares)
20 # Output: [1, 4, 9, 16, 25]
21
22
```

Output :

A screenshot of a terminal window showing the execution of the Python script. The command to run the script is entered, and the output [1, 4, 9, 16, 25] is displayed.

```
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe "c:/Users/sonu/Desktop/AI_Coding/Performance Optimization.py"
[1, 4, 9, 16, 25]
PS C:\Users\sonu\Desktop\AI_Coding>
```


Task Description #6 – Complexity Reduction Task: Use AI to simplify overly complex logic.

Sample Input Code: `def grade(score): if`

`score >= 90:`

`return "A" else: if`

`score >= 80:`

`return "B" else: if`

`score >= 70:`

`return "C" else: if`

`score >= 60:`

`return "D" else:`

`return "F"`

Expected Output:

- Cleaner logic using `elif` or dictionary mapping.

Code:

Complexity Reduction.py X

▶ ▢ ...

Complexity Reduction.py > ...

```
1  # Act as a Python code quality expert.
2  # Analyze the following Python code and simplify overly complex logic without changing its
3  # Your tasks:
4  # Identify unnecessarily nested conditions.
5  # Reduce complexity using cleaner control flow (like elif or better structures).
6  # Improve readability and maintainability.
7  # Keep the same output and grading logic.
8  # Follow clean coding best practices.
9  def grade_student(score):
10     if score >= 90:
11         return 'A'
12     elif score >= 80:
13         return 'B'
14     elif score >= 70:
15         return 'C'
16     elif score >= 60:
17         return 'D'
18     else:
19         return 'F'
20 # Example usage:
21 student_score = 85
22 grade = grade_student(student_score)
23 print(f"The student's grade is: {grade}")
24 # The original code is already quite straightforward and follows good practices. However,
25
26 def grade_student(score):
27     if score >= 90:
28         return 'A'
29     if score >= 80:
30         return 'B'
31     if score >= 70:
32         return 'C'
33     if score >= 60:
34         return 'D'
```

